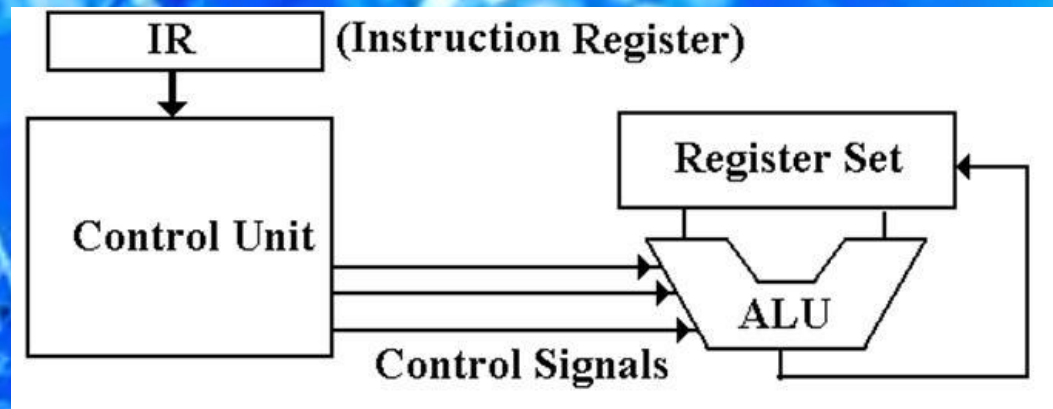
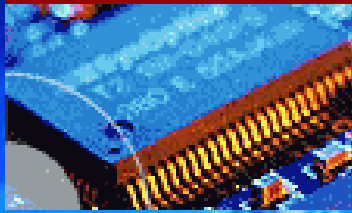




CPU Essentials

CPU Architecture & Datapath

Part2/2

Embedded Systems II

L19

Dr Simon Winberg



Electrical Engineering
University of Cape Town

Outline of Lecture

- Computer Design Fundamentals
- Building the Instruction Set
- The Function Unit in a CPU
- The ALU in a CPU
- Datapath pass 2: understanding the connection of the parts CU -> FU -> ALU
- Deciding the instruction set for a given processor design (PCP design activity)

Linked reading, for going deeper into these topics that are more introduced in these lectures:

Chapter 2 of Textbook: D. Patterson and J. Hennessy - Computer Organization and Design - ARM Edition.pdf

Computer Design Fundamentals

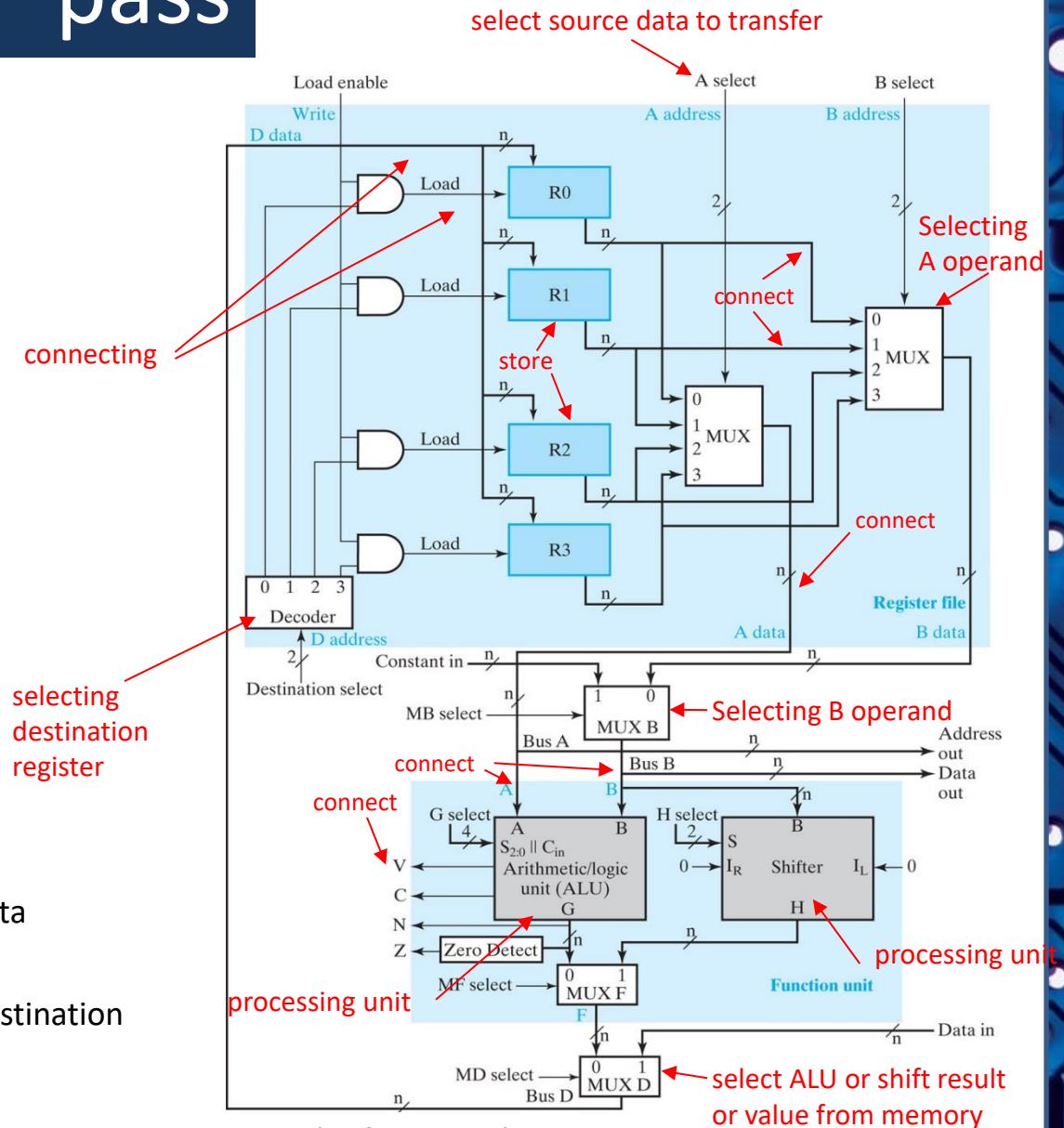
Datapath – 1st pass

Datapath design is largely about the circuitry to store data and to select and connect data to be transferred between processing units in a processor

Will briefly explain the parts...

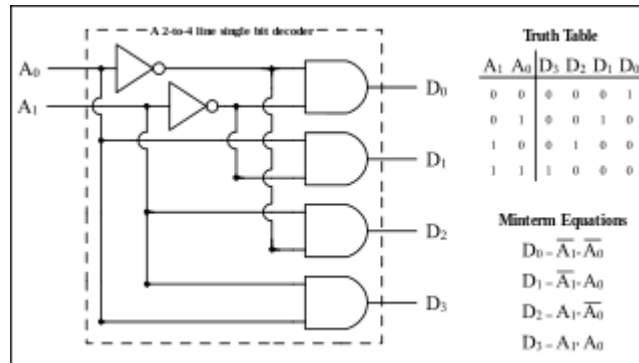
1. 4x registers available in this arch
2. Select destination & source for data
3. Connections between the parts
4. Move result or shifted value to destination

Onwards to....



Example of a Datapath

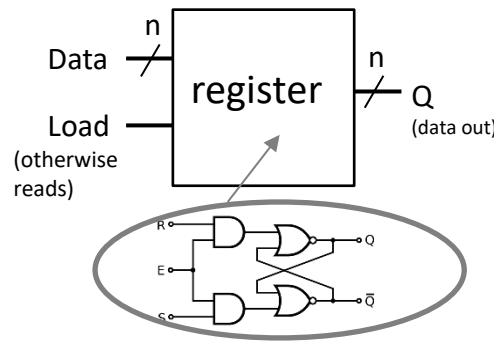
Brief refresher – processor pieces



The Decoder

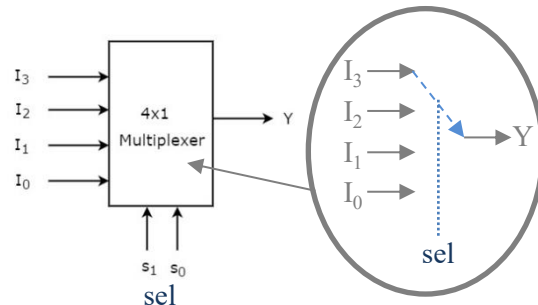
‘Fans out bits’

i.e. converts m-bit input to activating one of 2^m output signals. This is used for decoding an instruction, to chip select one of multiple operations



The Register

Stores values. Basically a D-type flip flop that can be enabled(load)/disabled(read)



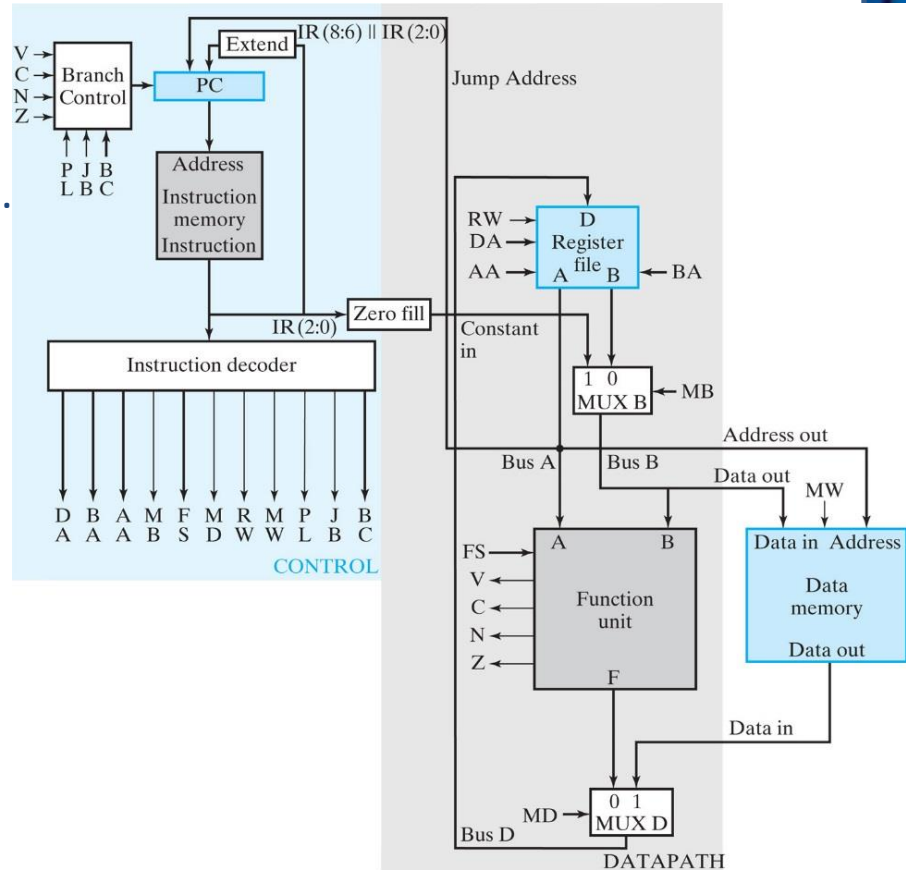
The Multiplexer

m-input sel line to select 1 of 2^m input lines (I) to connect through to the output line (Y)

Instruction Set Architecture (ISA)

General Computer Architecture comprises:

- Instruction Set Architecture* ...
- *Storage Resources:*
 - Instruction memory,
 - Data memory,
 - Register file,
 - Program counter
- *Datapath:*
 - Runs the instructions and activates processes

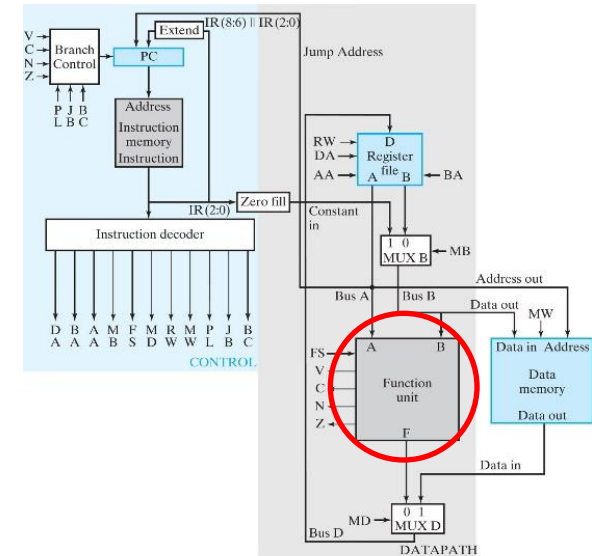


General Computer Architecture is designed around / comprises:

- Instruction Set Architecture*
- Storage Resources:
 - instruction memory,
 - data memory,
 - register file,
 - program counter
- Datapath: runs the instructions and activates processes

Instruction Set Architecture (ISA) =

- *Instruction format*: how the instructions are structured, how the bits of the instruction link to operations/data
- *Instruction specification*: describe each of the available instructions that the system can execute.



Block diagram of a single cycle computer

15	9	8	6	5	3	2	0
Opcode				Destination register (DR)		Source register A (SA)	
						Source register B (SB)	
Register							

An example instruction format that is likely used for arithmetic/logic operations on a RISC processor

The Function Unit

Function Unit (FU) (or 'execution unit')

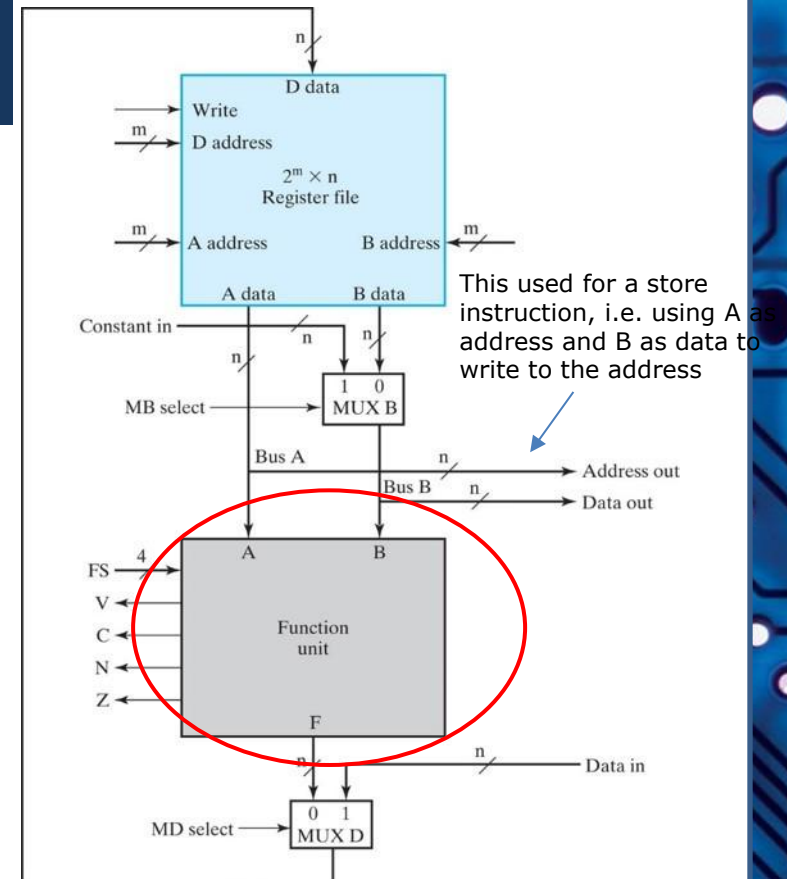
- Carries out the operations
- Could be structured in various ways.
- Typically...

$$FU = ALU + CU$$

and possibly

$$FU_{\text{main}} = \sum (FU_{\text{subunits}})$$

for a multi-processor system



Block diagram of a single cycle computer

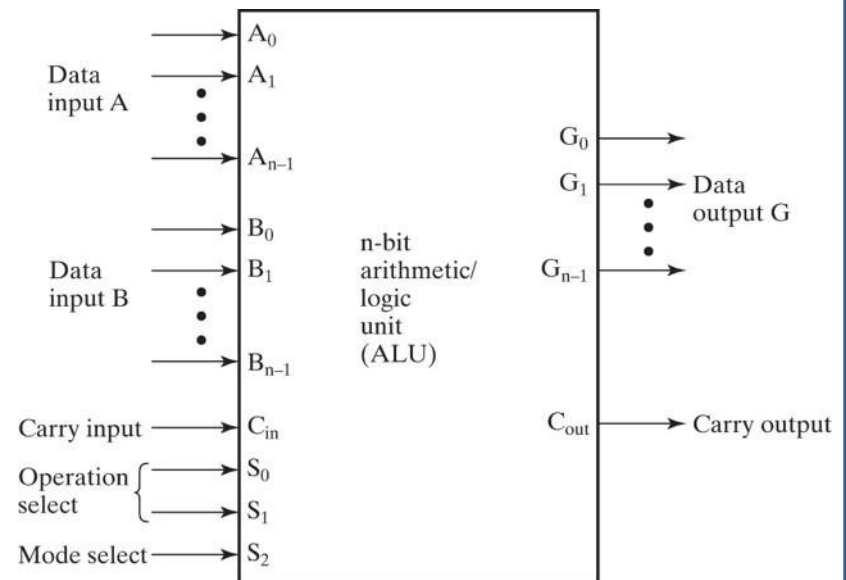
As show in this diagram, the FU has two input registers, A and B, that it operates on. There is a Function selector (FS) input used to chose the operation to perform. When this operation is performed, the result output is latched to F and updates status flags V,C,N,Z. The MD (memory data) select can bypass the FU if the instruction is loading from memory.

The ALU

- The Arithmetic/Logic Unit (ALU)
 - Arithmetic operations: + - * /
 - Logic operations: and, or, xor, not

The ALU has

- Inputs to provide data to be processed as well as
- Input for type of operation to be performed, (e.g. if it needs to do an ADD or AND, etc.)
- Has outputs that provide the results of the operation (G output in figure).
- Output flags, e.g. carry (C) output flag that stores status information of the operation (e.g. carry flag set) or error states (divide by zero).

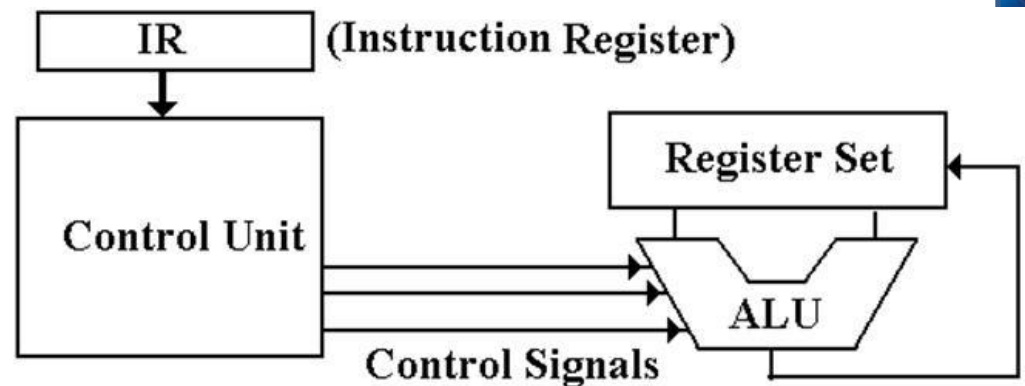


An n -Bit ALU

The Control Unit

- Control Unit (CU)
 - The CU is the part of the CPU that directs the working of the processor.
 - The CU coordinates the computer's memory, ALU and its input and output devices, making these parts respond to program instructions.

Sometimes the control unit is not shown in the design of a computer (or cannot be shown as a distinct subsystem) as it may be distributed through the system, **closely integrated within other components**; this is particularly the case for highly complex pipelined architectures where it is difficult to have a separate component to arbitrate the system.



Control Unit: can be abstracted as the circuitry used to sequence and control the system (this simplification obviously hides many aspects such as memory and IO)

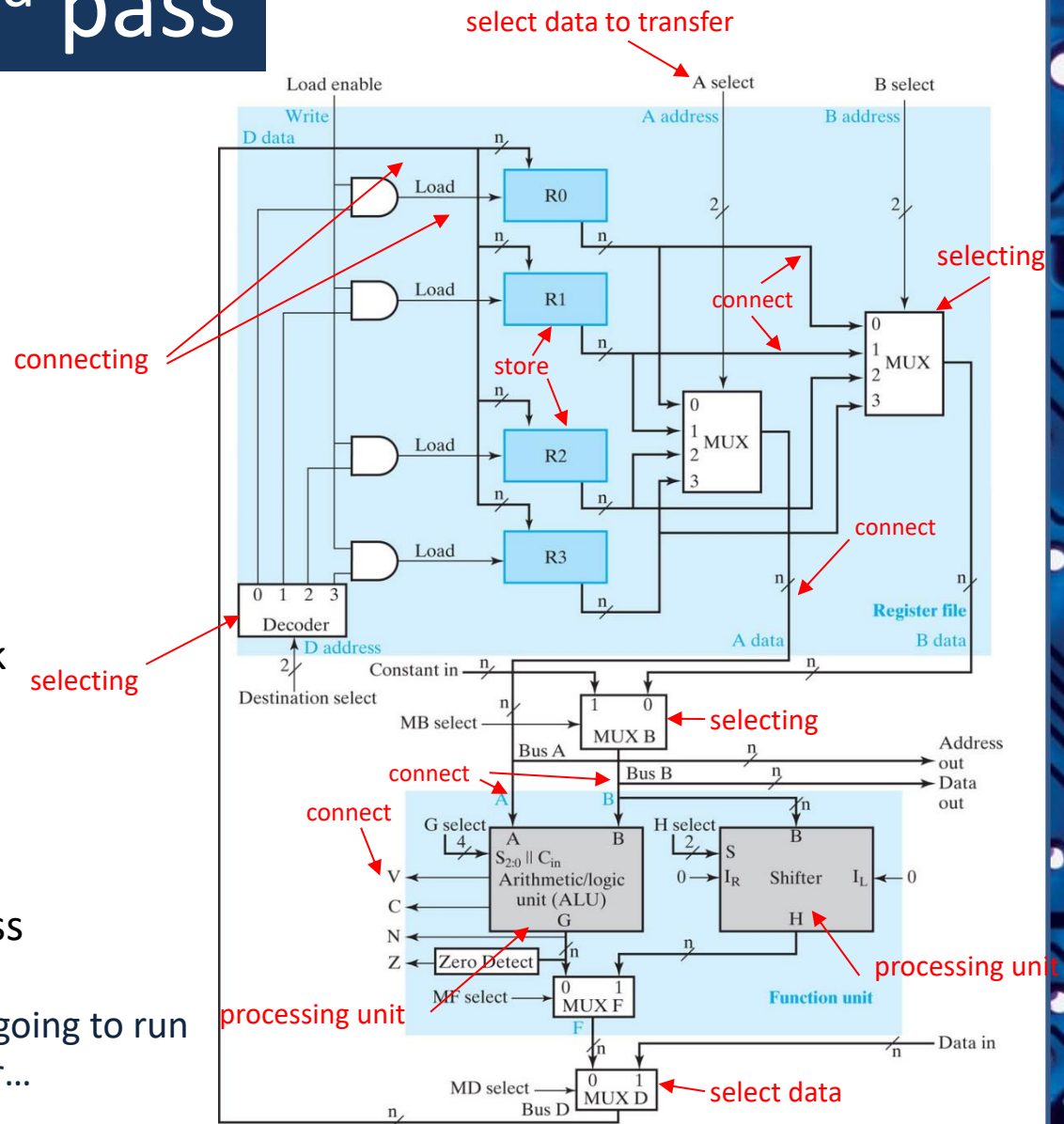
Datapath – 2nd pass

We've looked at the datapath to connect the pieces, the register file, the ALU, the FU, the CU.

Now you hopefully have an inkling of how the pieces work together to

- 1) get data to the FU inputs,
- 2) grab results from FU,
- 3) move FU results/memory to register or memory address

Don't be too worried yet: we are going to run through the process a little slower...



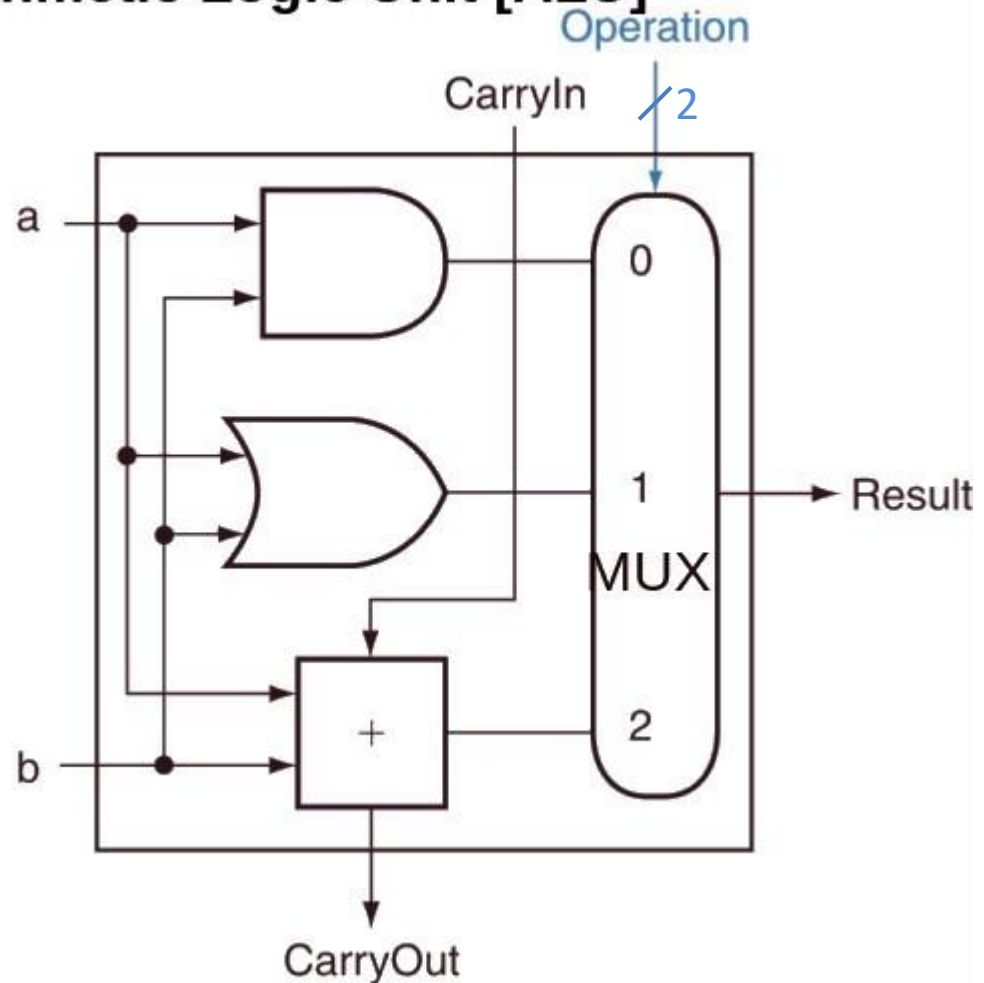
Example of a Datapath

Processor pieces

A simple ALU Example

Arithmetic Logic Unit [ALU]

This is a simplified ALU example, where there are two input operands (a,b), a carryIn bit and a carryOut bit. The Operation line (a 2 bit bus here) selects which operation (AND, OR or +) is applied.



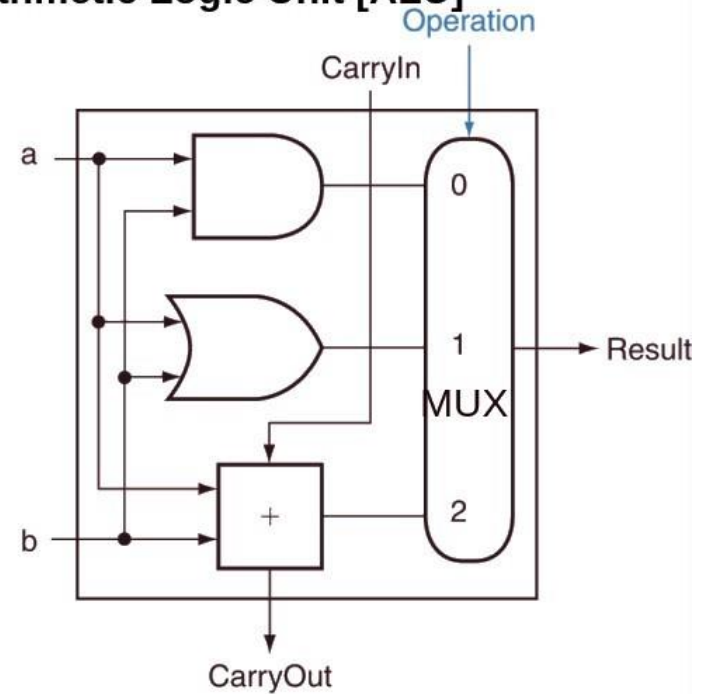
Deciding Processor Instructions

- You can either decide the instructions and their names first, or do that after you have the processor designed.
- Let us apply the second approach (not yet having named instructions) to illustrate the process of deciding instructions ...

Deciding Processor Instructions

- The ALU has only three operations: AND, OR, +
- Thus we don't yet know how many registers in the register bank, we don't yet need that.
- Let's just say there are 4 registers called R1, R2, R3, R4 these could link to a, b or Result
- ...

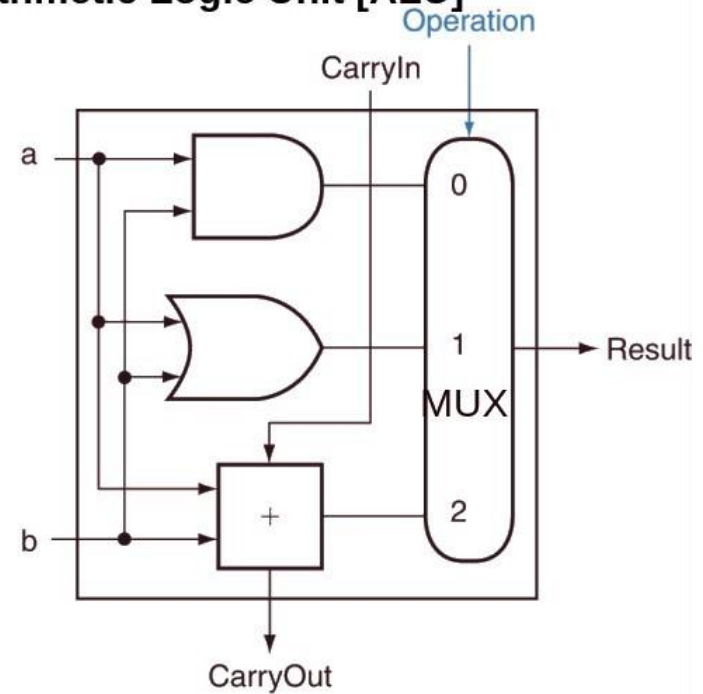
Arithmetic Logic Unit [ALU]



Deciding Processor Instructions

- We can then decide for this ISA, we have three instructions, that we can name: ADD, OR and AND
- Since they draw data from or move data to one of the registers, the instructions could be represented as:
AND R1, R2, R3 // $R1 = R2 \& R3$
OR R1, R2, R3 // $R1 = R2 | R3$
ADD R1, R2, R3 // $R1 = R2 + R3$

Arithmetic Logic Unit [ALU]

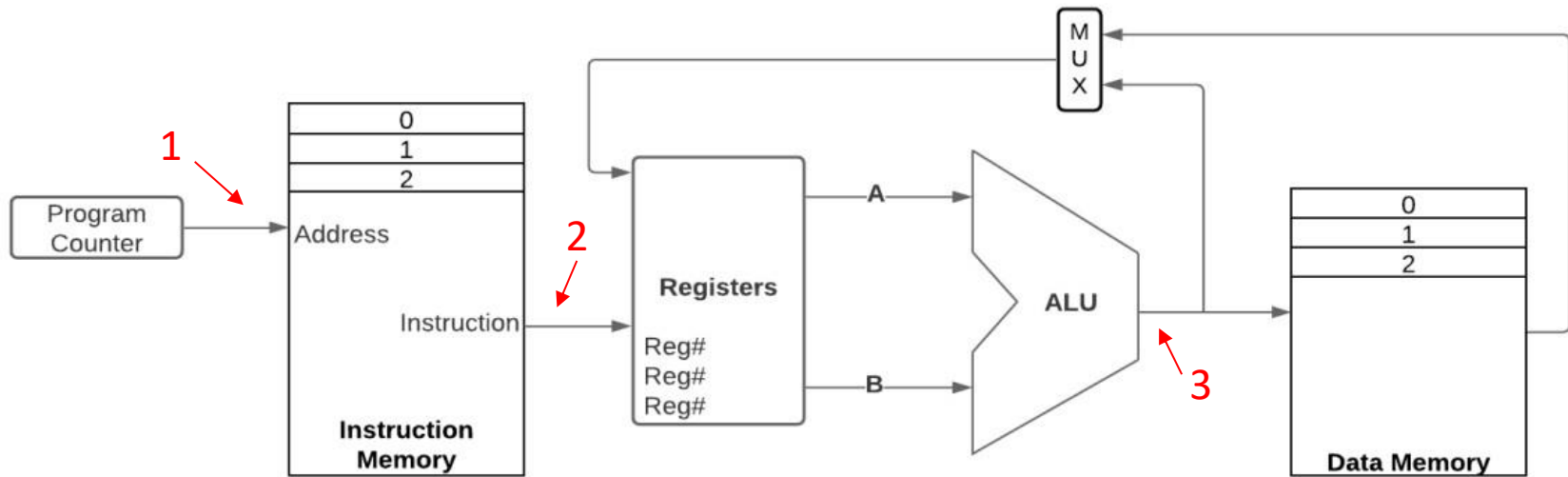


Connecting the parts

- The program, comprising instructions to run, is stored in the memory
- The PC (program counter) points to an address in memory where the next instruction is fetched
- The control unit coordinates the fetching of the next instruction from memory, and then selects source data to applied to the operation, and where the result of the operation is sent. (see next slide)

Basic Processor Overview

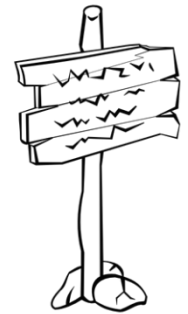
- Illustration of the main parts of a processor and sequencing of the data movement



1. Program Counter points to the Address of an Instruction in the Instruction Memory // using Modified Harvard Architecture in this example*
2. Instruction determines what values are placed on A & B operands for ALU
3. Output from the ALU stored in the Data Memory OR written back into a registers in the register bank

Where to later with ISA ...

- Getting into ARM processor fundamentals and RISC vs. CISC design.
- This builds on ISA



where to?

We look at real instructions for a standard ARM architecture, e.g.:

BRANCH instructions

J address @ jump to address

DATA PROCESSING instructions

ADD R1,R2,R3 @ $R1 = R2 + R3$

CMP R1,R2 @ compare R1 and R2

SWP R1,R2 @ swap value of R1 and R2

STATUS instructions

MSR @ move status register flags

LOAD and STORE instructions

LDR R1, address @ load from memory

STR R1, address @ store to memory

EXCEPTION instructions

SWI @ cause software interrupt

Plans for last two week of Term

- Solidifying your understanding of Computer Processor Architecture
- Memory and its integration with a processor
- [Miniscule processor design activity – *skip?*]
- Prepare for exam (prep task)
- Assembly and more complex issues of the ARM processor were covered 'Module2 ARM'

The Next Episode...

Lecture L20

- Concluding the course
- Going over practice and exam prep activities

